Understanding autoregressive models from a mathematical view

Pipi Hu

Beijing Institute of Mathematical Sciences and Applications

October 7, 2025

Feynman's words

What I cannot create, I do not understand.

Outline

What are autoregressive models?

Given a sequence $\mathbf{x}_{1:T} = (x_1, x_2, \ldots, x_T)$, autoregressive models factorize the joint probability as:

$$p(\mathbf{x}_{1:T}) = \prod_{t=1}^{T} p(x_t | \mathbf{x}_{t-1:1}) \tag{1}$$

where $\mathbf{x}_{t-1:1} = (x_{t-1}, x_{t-2}, \ldots, x_1)$ represents the history.
Key properties:

- Sequential generation: Generate one token at a time
- Conditional independence: Each token depends only on previous tokens
- Causal structure: No future information leakage

Mathematical foundation: Chain rule of probability

The autoregressive factorization follows directly from the chain rule of probability:

$$p(\mathbf{x}_{1:T}) = p(x_1) \prod_{t=2}^{T} p(x_t|\mathbf{x}_{t-1:1}) \tag{2}$$

This factorization is:

- **Always valid**: No assumptions required
- **Exact**: No approximation involved
- **Order-dependent**: Different orderings give different factorizations

The challenge is modeling the conditional distributions $p(x_t|\mathbf{x}_{t-1:1})$.

Why autoregressive models?

Advantages of autoregressive modeling:

1. **Simplicity**: Direct application of maximum likelihood
2. **Interpretability**: Clear probabilistic interpretation
3. **Flexibility**: Can model complex dependencies
4. **Training stability**: Well-understood optimization landscape

Challenges:

1. **Sequential generation**: Cannot parallelize during inference
2. **Error propagation**: Mistakes compound over time
3. **Long-range dependencies**: Hard to capture distant relationships

Outline

Maximum Likelihood Estimation

Given a dataset $\mathcal{D} = \{\mathbf{x}_{1:T}^{(i)}\}_{i=1}^{N}$, we maximize the log-likelihood:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \log p(\mathbf{x}_{1:T}^{(i)}; \boldsymbol{\theta}) \tag{3}$$

Substituting the autoregressive factorization:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \sum_{t=1}^{T} \log p(x_t^{(i)} | \mathbf{x}_{t-1:1}^{(i)}; \boldsymbol{\theta}) \tag{4}$$

This decomposes into independent optimization problems for each conditional distribution.

Cross-entropy loss and its interpretation

For discrete sequences, the loss becomes:

$$\mathcal{L}(\boldsymbol{\theta}) = -\sum_{i=1}^{N}\sum_{t=1}^{T} \log p(x_t^{(i)} | \mathbf{x}_{t-1:1}^{(i)}; \boldsymbol{\theta}) \tag{5}$$

This is equivalent to minimizing the cross-entropy:

$$H(p_{\text{data}}, p_{\boldsymbol{\theta}}) = \mathbb{E}_{\mathbf{x}_{1:T} \sim p_{\text{data}}} \left[-\sum_{t=1}^{T} \log p(x_t | \mathbf{x}_{t-1:1}; \boldsymbol{\theta}) \right] \tag{6}$$

Key insight: We're learning to predict the next token given the history, which is exactly what we need for generation.

Teacher forcing and training efficiency

During training, we use **teacher forcing**:

- Input: Ground truth history $\mathbf{x}_{t-1:1}$
- Target: Next token x_t
- Loss: Cross-entropy between predicted and true distributions

This allows:

1. **Parallel training**: All time steps computed simultaneously
2. **Stable gradients**: No error propagation during training
3. **Efficient optimization**: Standard backpropagation

Note: Training and inference use different procedures!

Outline

Outline

Recurrent Neural Networks (RNNs)

The simplest autoregressive architecture:

$$h_t = f(h_{t-1}, x_{t-1}; \boldsymbol{\theta}) \tag{7}$$

$$p(x_t|\mathbf{x}_{t-1:1}) = g(h_t; \boldsymbol{\theta}) \tag{8}$$

where h_t is the hidden state and f, g are neural networks.

Advantages:

- Natural sequential processing
- Parameter sharing across time
- Variable-length sequences

Limitations:

- Vanishing/exploding gradients
- Limited long-range dependencies
- Sequential computation

Outline

Long Short-Term Memory (LSTM)

LSTM addresses RNN limitations with gating mechanisms:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \text{(forget gate)} \tag{9}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{(input gate)} \tag{10}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad \text{(candidate values)} \tag{11}$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad \text{(cell state)} \tag{12}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \text{(output gate)} \tag{13}$$

$$h_t = o_t * \tanh(C_t) \quad \text{(hidden state)} \tag{14}$$

Key innovations:

- Gating: Control information flow
- Cell state: Long-term memory storage
- Gradient flow: Better backpropagation

Outline

From RNN/CNN limits to attention

- RNNs: sequential dependencies $\Rightarrow$ low parallelism; vanishing gradients for long-range context.
- CNNs: fixed receptive fields; long-range dependencies need many layers/dilations.
- **Transformer**: *self-attention* provides content-based, global interactions with full parallelism.
- Complexity per layer: RNN $\mathcal{O}(n)$ but sequential; self-attention $\mathcal{O}(n^2)$ but fully parallel and expressive.

Scaled Dot-Product Attention: Key formula

Inputs: queries $Q \in \mathbb{R}^{n \times d_k}$, keys $K \in \mathbb{R}^{n \times d_k}$, values $V \in \mathbb{R}^{n \times d_v}$.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V, \tag{15}$$

where M is an optional mask (e.g., $M_{ij} = -\infty$ for $j > i$ in autoregressive decoding).

- Scaling by $\sqrt{d_k}$ keeps gradients in a reasonable range.
- **Interpretation**: each token re-weights all values using similarity to keys.

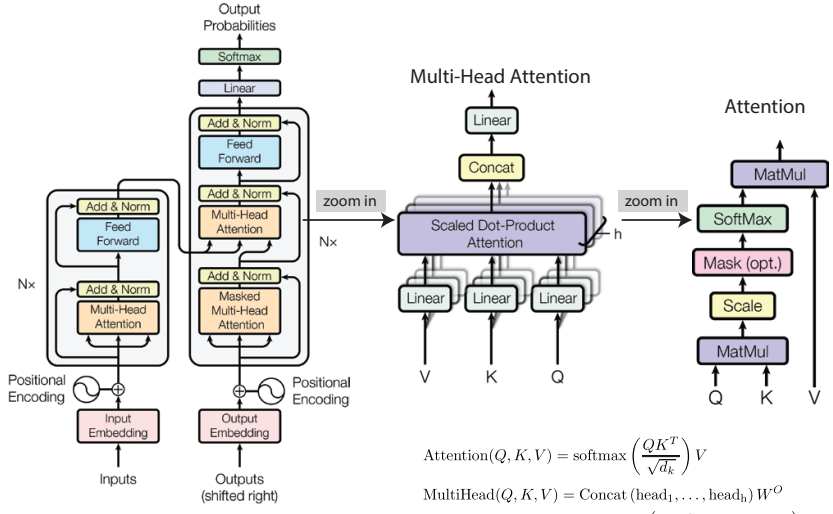
Multi-Head Attention (MHA)

$$\mathrm{MHA}(X) = \mathrm{Concat}(H_1, \ldots, H_h)\, W^O, \tag{16}$$

$$H_i = \mathrm{Attention}(XW_i^Q,\, XW_i^K,\, XW_i^V), \quad i = 1, \ldots, h. \tag{17}$$

- Heads attend to different subspaces/patterns.
- Residual + LayerNorm around MHA and FFN blocks stabilize training.

Transformer macro-architecture



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}\left(\text{head}_1, \ldots, \text{head}_\text{h}\right)W^O$$

$$\text{where } \text{head}_i = \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right)$$

Outline

Greedy decoding

The simplest generation strategy:

$$x_t = \arg\max_x p(x|\mathbf{x}_{t-1:1}; \boldsymbol{\theta}) \tag{18}$$

Properties:

- Deterministic: Same input always produces same output
- Fast: Single forward pass per token
- Often suboptimal: Local decisions may not be globally optimal

Issues:

- Repetition: May get stuck in loops
- Lack of diversity: Always chooses most likely token
- Mode collapse: May not explore the full distribution

Random sampling

Sample from the full distribution:

$$x_t \sim p(x|\mathbf{x}_{t-1:1}; \boldsymbol{\theta}) \tag{19}$$

Advantages:

- Diversity: Explores the full distribution
- Natural: Reflects model uncertainty
- Flexibility: Can control randomness

Challenges:

- Quality control: May generate low-quality sequences
- Consistency: Less predictable output
- Evaluation: Harder to assess quality

Temperature scaling and nucleus sampling

Temperature scaling:

$$p_{\text{temp}}(x|\mathbf{x}_{t-1:1}) = \frac{\exp(f(x, \mathbf{x}_{t-1:1})/\tau)}{\sum_{x'} \exp(f(x', \mathbf{x}_{t-1:1})/\tau)} \tag{20}$$

where $\tau > 0$ controls randomness:

- $\tau \to 0$: Greedy decoding
- $\tau = 1$: Original distribution
- $\tau > 1$: More random

Nucleus sampling (Holtzman et al., 2019):

$$\text{Select smallest } V \text{ such that } \sum_{x \in V} p(x|\mathbf{x}_{t-1:1}) \geq p \tag{21}$$

Then sample from the truncated distribution.

Outline

Evaluation metrics for autoregressive models

Perplexity:

$$\text{Perplexity} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log p(x_t|\mathbf{x}_{t-1:1};\boldsymbol{\theta})\right) \tag{22}$$

Other metrics:

- BLEU: N-gram overlap with reference
- ROUGE: Recall-oriented evaluation
- BERTScore: Semantic similarity using BERT
- Human evaluation: Subjective quality assessment

Challenges:

- No single metric: Different tasks need different metrics
- Reference dependence: Many metrics need reference text
- Human evaluation: Expensive and subjective

Information-theoretic perspective

The entropy of an autoregressive model is:

$$H(p_\theta) = \mathbb{E}_{\mathbf{x}_{1:T} \sim p_\theta} \left[-\sum_{t=1}^{T} \log p(x_t | \mathbf{x}_{t-1:1}; \boldsymbol{\theta}) \right] \tag{23}$$

Key insights:

- Conditional entropy: $H(X_t|X_{t-1:1}) \leq H(X_t)$
- Chain rule: $H(X_{1:T}) = \sum_{t=1}^{T} H(X_t|X_{t-1:1})$
- Compression: Autoregressive models compress information

Compression ratio:

$$\text{Compression} = \frac{H(X_{1:T})}{T \log V} = \frac{\sum_{t=1}^{T} H(X_t|X_{t-1:1})}{T \log V} \tag{24}$$

Lower compression ratio means better modeling of dependencies.

Large language models and scaling laws

Scaling laws (Kaplan et al., 2020):

$$L(N, D) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D} \tag{25}$$

where N is model size, D is dataset size, and L is loss.

Key findings:

- Power law scaling: Performance improves predictably with scale
- Emergent abilities: New capabilities emerge at certain scales
- Efficient scaling: Larger models are more sample-efficient

Implications:

- Compute requirements: Training costs scale dramatically
- Data requirements: Need massive datasets
- Architecture choices: Transformer scales well

Common challenges and limitations

Training challenges:

1. Exposure bias: Training uses ground truth, inference uses model predictions
2. Teacher forcing mismatch: Different distributions during training and inference
3. Long sequences: Gradient vanishing/exploding

Generation challenges:

1. Repetition: Models may repeat phrases or sentences
2. Coherence: Long-range dependencies hard to maintain
3. Diversity vs. Quality: Trade-off between exploration and exploitation

Computational challenges:

1. Sequential generation: Cannot parallelize inference
2. Memory requirements: Need to store full history
3. Scalability: Training large models is expensive

Outline

Open questions

If you are interested in these questions, please feel free to write an email to me (pisquare@microsoft.com) with your comments.

1. How can we design better non-autoregressive models that maintain quality while enabling parallel generation?
2. What are the fundamental limits of autoregressive modeling? Can we prove lower bounds on the required model size for certain tasks?
3. How do we balance the trade-off between generation speed and quality in practical applications?
4. What role do autoregressive models play in the broader landscape of generative AI, and how do they compare to diffusion models and other approaches?
5. Can we develop theoretical frameworks to understand when autoregressive models will succeed or fail?

北京雁栖湖
应用数学研究院
Beijing Institute
of Mathematical
Sciences and Applications

Welcome inputs

Any comments?

Welcome your inputs!